

Homework 04: Sample solution

QMIR

September 30, 2026

Table of contents

1	Read and explore	1
2	Tidy	4
3	Transform	6
4	Join	8
4.1	Party families	8
4.2	EU membership	10
5	Count and summarise	12
6	Your pipeline: a data summary	16
	Reflection	19
	Session info and execution time	19

1 Read and explore

This homework uses **ParlGov**, a database of parties, elections and cabinets in 37 democracies. The file `data/parlgov.xlsx` is an Excel workbook with several sheets. `readxl` was installed with the tidyverse but is not loaded by it, so call its functions as `readxl::read_excel()`, as in class.

1. List the sheets of the workbook with `readxl::excel_sheets()`. Build the path with `here("data", "parlgov.xlsx")`.

```
path <- here("data", "parlgov.xlsx")
readxl::excel_sheets(path)
```

```
[1] "party"      "election" "cabinet"  "variable" "readme"
```

The workbook has five sheets: party, election, cabinet, variable and readme.

2. Read the election sheet into an object called elections and the party sheet into parties. The sheet = argument of readxl::read_excel() picks the sheet.
3. glimpse() both.

```
elections <- readxl::read_excel(path, sheet = "election")
parties <- readxl::read_excel(path, sheet = "party")

glimpse(elections)
```

```
Rows: 9,016
Columns: 16
$ country_name_short      <chr> "AUS", "AUS", "AUS", "AUS", "AUS", "AU~
$ country_name            <chr> "Australia", "Australia", "Australia",~
$ election_type           <chr> "parliament", "parliament", "parliamen~
$ election_date           <chr> "1901-03-30", "1901-03-30", "1901-03-
3~
$ vote_share              <dbl> 44.4, 34.2, 19.4, 1.4, 0.6, 29.7, 34.4~
$ seats                  <dbl> 32, 26, 15, 1, 1, 26, 25, 23, 1, 0, 27~
$ seats_total             <dbl> 75, 75, 75, 75, 75, 75, 75, 75, 75, 75~
$ party_name_short       <chr> "PP", "FTP", "ALP", "none", "one-
seat"~
$ party_name              <chr> "Protectionist Party", "Free Trade Par~
$ party_name_english      <chr> "Protectionist Party", "Free Trade Par~
$ left_right              <dbl> 7.4000, 6.0000, 3.8833, NA, NA, 7.4000~
$ country_id              <dbl> 33, 33, 33, 33, 33, 33, 33, 33, 33, 33~
$ election_id             <dbl> 731, 731, 731, 731, 731, 730, 730, 730~
$ previous_parliament_election_id <dbl> NA, NA, NA, NA, NA, 731, 731, 731, 731~
$ previous_cabinet_id     <dbl> NA, NA, NA, NA, NA, 997, 997, 997, 997~
$ party_id               <dbl> 1898, 1938, 1253, 1396, 2299, 1898, 19~
```

```
glimpse(parties)
```

Rows: 1,707

Columns: 23

```
$ country_name_short <chr> "AUS", "AUS", "AUS", "AUS", "AUS", "AUS", "AUS", "A~
$ country_name       <chr> "Australia", "Australia", "Australia", "Australia",~
$ party_name_short   <chr> "ACP-V", "AD", "AG", "ALP", "AP", "C", "CDP", "CLP"~
$ party_name_english <chr> "Australian Country Party (Vic)", "Australian Democ~
$ party_name         <chr> "Australian Country Party (Vic)", "Australian Democ~
$ party_name_ascii   <chr> "Australian Country Party (Vic)", "Australian Democ~
$ family_name_short  <chr> "con", "soc", "eco", "soc", "lib", "con", "chr", "c~
$ family_name        <chr> "Conservative", "Social democracy", "Green/Ecologis~
$ left_right         <dbl> 7.4000, 3.7004, 1.5439, 3.8833, 6.0000, 7.4000, 6.2~
$ state_market       <dbl> 6.4000, 3.3684, 1.2982, 4.0351, 6.7000, 6.4000, 5.7~
$ liberty_authority  <dbl> 6.9000, 1.1579, 0.5614, 3.2632, 3.6000, 6.9000, 7.1~
$ eu_anti_pro        <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ cmp               <dbl> NA, 63321, NA, 63320, NA, NA, NA, NA, NA, NA, NA, NA, 6~
$ euprofiler         <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ ees               <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ castles_mair       <dbl> NA, 102, NA, 101, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ huber_inglehart    <dbl> NA, 203, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, 204, N~
$ ray               <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ benoit_laver       <dbl> NA, 4703, 4739, 4721, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ chess             <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,~
$ country_id         <dbl> 33, 33, 33, 33, 33, 33, 33, 33, 33, 33, 33, 33, 33,~
$ party_id           <dbl> 1935, 120, 751, 1253, 1760, 1428, 1650, 2285, 1922,~
$ family_id          <dbl> 26, 11, 19, 11, 6, 26, 3, 26, 14, 26, 6, 11, 6, 26,~
```

4. Answer in prose: how many rows does each table have, and what is one row of each?

`elections` has 9016 rows and 16 columns, and one row is **one party in one election**. `parties` has 1707 rows and 23 columns, and one row is **one party**.

5. Which type did `election_date` get? In class, `read_csv()` read `accession_date` as a `<date>`. Why do you think the two differ?

`election_date` is a `<chr>`, plain text. `read_csv()` reads text and guesses a type from how the text looks, so 2004-05-01 became a date. `readxl::read_excel()` instead keeps the type the cell has in Excel, and in this workbook the dates are stored as text cells. The lesson is the same either way: `glimpse()` new data and check every type before using it.

6. `count()` the values of `election_type`. What are the two types, and which is more common?

```
elections |>
  count(election_type)
```

```
# A tibble: 2 x 2
  election_type      n
  <chr>          <int>
1 ep             1702
2 parliament     7314
```

The two types are `parliament`, national parliament elections, and `ep`, elections to the European Parliament. National elections are more common, 7314 rows against 1702.

7. The `variable` sheet documents every column. Read it and find out what `left_right` measures and on which scale.

```
variables <- readxl::read_excel(path, sheet = "variable")
variables |>
  filter(name == "left_right") |>
  select(table_name, description)
```

```
# A tibble: 3 x 2
  table_name      description
  <chr>          <chr>
1 view_party     0--10 scale mean value in left/right dimension with data from C~
2 view_election 0--10 scale mean value in left/right dimension with data from C~
3 view_cabinet  0--10 scale mean value in left/right dimension with data from C~
```

`left_right` is a party's position on the left-right dimension, on a scale from 0 (left) to 10 (right). It is the mean of several expert surveys (Castles and Mair, Huber and Inglehart, Benoit and Laver, and the Chapel Hill survey), so it is one position per party, not per election.

2 Tidy

1. Is `elections` tidy? Say in one sentence what a row is, and check it against the three rules of tidy data.

Yes. A row is one party in one election, every column is one variable (country, date, vote share, seats, and so on), and every cell holds one value.

2. `parties` holds three positions of every party, each on a scale from 0 to 10, in three columns: `left_right`, `state_market` and `liberty_authority`. For a question such as “how do the party families differ on each scale”, the scale is itself a variable. **Before running anything**, predict how many rows a tibble with one row per party and scale will have.

There are 1707 parties and three scales, so the long tibble has $1707 \times 3 = 5121$ rows.

3. Pivot the three columns longer, with the names going to `scale` and the values to `position`, and assign the result to `positions`. Check your prediction with `nrow()`.

```
positions <- parties |>
  pivot_longer(
    cols = c(left_right, state_market, liberty_authority),
    names_to = "scale",
    values_to = "position"
  )
nrow(positions)
```

```
[1] 5121
```

The prediction holds: 5121 rows.

4. Compute the mean `position` per `scale` with one `group_by()` and `summarise()`. The first attempt returns NA: you met the fix in homework 03.

```
positions |>
  group_by(scale) |>
  summarise(mean_position = mean(position, na.rm = TRUE))
```

```
# A tibble: 3 x 2
  scale          mean_position
  <chr>          <dbl>
1 left_right      5.42
2 liberty_authority 5.18
3 state_market    4.96
```

Without `na.rm = TRUE`, every mean is NA, because about 320 parties have no expert placement. The average party sits a little right of the midpoint on `left_right` and a little left of it on `state_market`.

5. **Stretch:** pivot `positions` back to wide, and check that you get the columns you started from.

```
positions |>
  pivot_wider(names_from = scale, values_from = position) |>
  select(party_id, left_right, state_market, liberty_authority)
```

```
# A tibble: 1,707 x 4
  party_id left_right state_market liberty_authority
    <dbl>    <dbl>    <dbl>         <dbl>
1    1935      7.4      6.4          6.9
2     120      3.70     3.37          1.16
3     751      1.54     1.30          0.561
4    1253      3.88     4.04          3.26
5    1760       6      6.7          3.6
6    1428      7.4      6.4          6.9
7    1650      6.2     5.7          7.1
8    2285      7.4      6.4          6.9
9     1922      1.3     1.4           3
10    1914      7.4      6.4          6.9
# i 1,697 more rows
```

Pivoting back returns the three original columns with the original 1707 rows.

3 Transform

From here on, work with **national parliament elections** only.

1. Filter elections to the rows with `election_type == "parliament"` and assign the result to `parl`. How many rows remain?
2. In the same pipeline, add three columns with one `mutate()`:
 - `year`, the year of the election. `election_date` is text, so turn it into a date first: `year(as.Date(election_date))`.
 - `seat_share`, the percentage of seats a party won: `seats / seats_total * 100`.
 - `disproportionality`, how far a party's seat share lies from its vote share, in either direction: `abs(seat_share - vote_share)`. `abs()` returns the absolute value, so a party 5 points over and a party 5 points under both count as 5.

```
parl <- elections |>
  filter(election_type == "parliament") |>
  mutate(
    year = year(as.Date(election_date)),
    seat_share = seats / seats_total * 100,
    disproportionality = abs(seat_share - vote_share)
  )
nrow(parl)
```

```
[1] 7314
```

7314 rows remain, from 845 national elections in 37 countries.

3. Show the most recent German election in `parl`, with only the columns `party_name_english`, `vote_share`, `seats` and `seat_share`, sorted by vote share with the largest party first. Hint: find the date first, then `filter()` on it.

```
germany_last <- parl |>
  filter(country_name == "Germany") |>
  pull(election_date) |>
  max()
germany_last
```

```
[1] "2021-09-26"
```

```
parl |>
  filter(country_name == "Germany", election_date == germany_last) |>
  select(party_name_english, vote_share, seats, seat_share) |>
  arrange(desc(vote_share))
```

```
# A tibble: 12 x 4
```

	party_name_english <chr>	vote_share <dbl>	seats <dbl>	seat_share <dbl>
1	Social Democratic Party of Germany	25.7	206	28.0
2	Christian Democratic Union	18.9	152	20.7
3	Alliance 90 / Greens	14.8	118	16.0
4	Free Democratic Party	11.5	92	12.5
5	Alternative for Germany	10.3	83	11.3
6	Christian Social Union	5.17	45	6.11
7	PDS The Left	4.89	39	5.30
8	Free Voters	2.43	0	0
9	Animal Protection Party	1.45	0	0
10	Grassroots Democratic Party of Germany	1.36	0	0
11	The PARTY	0.99	0	0
12	South Schleswig Voter League	0.12	1	0.136

The most recent German election in the data is the Bundestag election of 26 September 2021.

4. In one or two sentences: which parties won more seat share than vote share, and which won no seats at all despite winning votes? What rule of the German electoral system do you think explains the second group?

Every party that entered the Bundestag won a slightly larger seat share than vote share, because the votes of the parties that stayed out are not turned into seats at all. The Free Voters, the Animal Protection Party, dieBasis and Die PARTEI won votes but no seats: they fell below the **5 percent threshold**. Two exceptions show how the threshold works. The Left won seats with 4.9 percent because it won three constituencies directly, which exempts a party from the threshold, and the South Schleswig Voter League, a party of the Danish minority, is exempt by law.

5. **Stretch:** find the five largest seat shares in the whole of `parl`, with the country, the date, the party, its vote share and its seat share. What do the countries have in common?

```
parl |>
  arrange(desc(seat_share)) |>
  select(country_name, election_date, party_name_english, vote_share, seat_share) |>
  slice(1:5)
```

```
# A tibble: 5 x 5
  country_name election_date party_name_english vote_share seat_share
  <chr>         <chr>         <chr>                <dbl>     <dbl>
1 Canada       1958-03-31    Progressive Conservative P~ 53.7      78.5
2 United Kingdom 1931-10-27    Conservatives          55.0      76.4
3 Canada       1984-09-04    Progressive Conservative P~ 50.0      74.8
4 Canada       1940-03-26    Liberal Party of Canada    51.5      73.9
5 Greece       1974-11-17    New Democracy           54.4      73.3
```

All five are Canada, the United Kingdom and Greece. Each turned a vote share of around 50 percent into three quarters of the seats. Canada and the United Kingdom elect their parliaments by plurality in single-member districts, and Greece used a proportional system with a large seat bonus for the winner. Such rules hand the largest party a majority of seats it did not win in votes.

4 Join

4.1 Party families

`parl` does not know which party family a party belongs to. `parties` does, in `family_name`, and both tables share the key `party_id`.

1. **Before running anything**, predict how many rows `parl` will have after a `left_join()` that adds `family_name` from `parties`.

`party_id` is unique in `parties`, so every row of `parl` finds at most one partner, and `left_join()` keeps every row of `parl`. The prediction is 7314 rows, the same as `parl`.

2. Join `parties` `|> select(party_id, family_name)` onto `parl` by `party_id`, and assign the result to `parl_family`. Check your prediction, and count the NAs in `family_name`.

```
parl_family <- parl |>
  left_join(parties |> select(party_id, family_name), by = "party_id")

nrow(parl_family)
```

```
[1] 7314
```

```
sum(is.na(parl_family$family_name))
```

```
[1] 0
```

The prediction holds, and no row has a missing `family_name`: every party in `parl` is in `parties`.

3. Why select the two columns before the join? Try the join once with the whole of `parties` and look at the column names.

```
parl |>
  left_join(parties, by = "party_id") |>
  names()
```

[1] "country_name_short.x"	"country_name.x"
[3] "election_type"	"election_date"
[5] "vote_share"	"seats"
[7] "seats_total"	"party_name_short.x"
[9] "party_name.x"	"party_name_english.x"
[11] "left_right.x"	"country_id.x"
[13] "election_id"	"previous_parliament_election_id"
[15] "previous_cabinet_id"	"party_id"
[17] "year"	"seat_share"
[19] "disproportionality"	"country_name_short.y"
[21] "country_name.y"	"party_name_short.y"
[23] "party_name_english.y"	"party_name.y"
[25] "party_name_ascii"	"family_name_short"
[27] "family_name"	"left_right.y"

```

[29] "state_market"           "liberty_authority"
[31] "eu_anti_pro"            "cmp"
[33] "euprofiler"             "ees"
[35] "castles_mair"           "huber_inglehart"
[37] "ray"                    "benoit_laver"
[39] "chess"                  "country_id.y"
[41] "family_id"

```

Without the `select()`, the join adds all 22 other columns of `parties`. Columns that exist in both tables, such as `country_name`, `party_name_english` and `left_right`, appear twice with the suffixes `.x` (from `parl`) and `.y` (from `parties`). `country_name` no longer exists under that name, so every later pipeline that uses it breaks. Selecting the key and the one column you need keeps the result clean.

4.2 EU membership

The EU membership file from class also lives on GitHub. Read it straight from the web:

```

base <- "https://raw.githubusercontent.com/qmir-2026-fall/qmir-2026-fall.github.io/main/webs
eu_members <- read_csv(paste0(base, "eu-members.csv"))

```

The key has a different name in each table: `country_name` in `ParlGov` and `country` in `eu_members`. `by = join_by(country_name == country)` tells `left_join()` that the two columns are the same key.

1. Build a table of ParlGov's countries with one row each: `parl |> count(country_name)`. Then join `eu_members` onto it.

```

countries <- parl |>
  count(country_name) |>
  left_join(eu_members, by = join_by(country_name == country))

countries

```

```

# A tibble: 37 x 4
  country_name      n accession_date exit_date
  <chr>          <int> <date>         <date>
1 Australia      295 NA              NA
2 Austria        157 1995-01-01     NA
3 Belgium        331 1958-01-01     NA
4 Bulgaria       132 2007-01-01     NA

```

```

5 Canada          204 NA          NA
6 Croatia         105 2013-07-01  NA
7 Cyprus          75  2004-05-01  NA
8 Czech Republic  103 2004-05-01  NA
9 Denmark         464 1973-01-01  NA
10 Estonia         80  2004-05-01  NA
# i 27 more rows

```

2. **Before running the join**, predict how many rows the result has.

The left table has one row per ParlGov country, 37 rows, and `left_join()` keeps exactly those.

3. How many countries get NA in `accession_date`? `filter()` them with `is.na(accession_date)` and read the list. One of them is an EU member state. Which one, and why did the join miss it?

```
sum(is.na(countries$accession_date))
```

```
[1] 10
```

```
countries |>
  filter(is.na(accession_date)) |>
  select(country_name)
```

```

# A tibble: 10 x 1
  country_name
  <chr>
1 Australia
2 Canada
3 Iceland
4 Israel
5 Japan
6 New Zealand
7 Norway
8 Slovakia
9 Switzerland
10 Turkey

```

10 countries get NA. Nine of them are genuinely outside the EU: Australia, Canada, Iceland, Israel, Japan, New Zealand, Norway, Switzerland and Turkey. **Slovakia** is a member since 2004. ParlGov spells it **Slovakia** and the EU file spells it **Slovak Republic**, so the keys never match, and the join returns NA without any error.

4. Fix the spelling in `eu_members` so that it matches ParlGov, and overwrite `eu_members` with the fixed version. Hint: `mutate(country = if_else(country == "...", "...", country))` replaces one value and keeps every other.

```
eu_members <- eu_members |>
  mutate(country = if_else(country == "Slovak Republic", "Slovakia", country))
```

5. Run the join again. Show with `count()` that 28 countries now have an accession date and 9 do not.

```
parl |>
  count(country_name) |>
  left_join(eu_members, by = join_by(country_name == country)) |>
  count(has_accession_date = !is.na(accession_date))
```

```
# A tibble: 2 x 2
  has_accession_date      n
  <lgl>                <int>
1 FALSE                  9
2 TRUE                  28
```

28 countries now have an accession date, all current members plus the United Kingdom, and 9 do not.

Warning

A join matches text exactly. A spelling difference gives NA without any error or warning. Count the NAs after every join, and read the list of rows that failed to match.

5 Count and summarise

1. Count the rows of `parl_family` per `family_name`, largest first. Two of the values are not party families at all. Which ones, and what do they mean?

```
parl_family |>
  count(family_name, sort = TRUE)
```

```
# A tibble: 11 x 2
  family_name      n
  <chr>          <int>
1 Social democracy 1304
2 Conservative    1248
3 Liberal         1118
4 Communist/Socialist 803
5 Christian democracy 661
6 Right-wing      557
7 no family       492
8 Special issue   453
9 Green/Ecologist 334
10 Agrarian       305
11 to be coded    39
```

Social democracy, Conservative and Liberal are the largest families. `no family` and `to be coded` are not families. `no family` collects rows ParlGov cannot place in any family, such as independents, the pooled “other parties” of an election, and electoral alliances. `to be coded` marks parties ParlGov has not classified yet. Both have to be dropped before comparing families.

2. For every country, compute the mean `disproportionality` and the number of rows with `n()`. Sort so that the most disproportional country comes first. `vote_share` has missing values, so remember `na.rm = TRUE`.

```
disprop <- parl |>
  group_by(country_name) |>
  summarise(
    mean_disprop = mean(disproportionality, na.rm = TRUE),
    n = n()
  ) |>
  arrange(desc(mean_disprop))

disprop |>
  print(n = 37)
```

```
# A tibble: 37 x 3
  country_name mean_disprop      n
  <chr>         <dbl> <int>
1 Canada       5.16    204
2 Turkey       4.78     84
3 New Zealand  4.37    198
```

4	France	4.20	286
5	Australia	4.04	295
6	Japan	3.69	221
7	United Kingdom	3.63	274
8	Hungary	3.62	93
9	Greece	3.07	142
10	Poland	2.72	118
11	Lithuania	2.64	121
12	Czech Republic	2.64	103
13	Slovakia	2.61	117
14	Luxembourg	2.60	164
15	Iceland	2.52	196
16	Norway	2.45	251
17	Bulgaria	2.34	132
18	Portugal	2.32	145
19	Croatia	2.20	105
20	Romania	2.15	106
21	Latvia	2.09	107
22	Estonia	1.84	80
23	Cyprus	1.77	75
24	Ireland	1.74	226
25	Malta	1.70	73
26	Slovenia	1.58	136
27	Spain	1.53	199
28	Austria	1.50	157
29	Belgium	1.34	331
30	Switzerland	1.28	362
31	Finland	1.24	287
32	Italy	1.11	275
33	Sweden	1.06	234
34	Denmark	0.986	464
35	Germany	0.802	285
36	Israel	0.593	334
37	Netherlands	0.423	334

3. In two or three sentences: which countries top the list, and which close it? What do you know about how these countries turn votes into seats?

The list opens with Canada, Turkey, New Zealand, France, Australia, Japan and the United Kingdom. Most of them elect all or part of their parliament in single-member districts: plurality in Canada, the United Kingdom and New Zealand before 1996, two rounds in France, the alternative vote in Australia. Turkey is the exception, and there the cause is a 10 percent national threshold that kept many parties out. The list closes with the Netherlands, Israel,

Germany, Denmark and Sweden, which elect by proportional representation in very large districts or with compensatory seats, so seat shares track vote shares closely. The mean over all parties is only a rough measure, but it separates majoritarian from proportional systems cleanly.

4. **Stretch:** is the largest party getting smaller over time? For each election, find the vote share of the largest party with `max()`, then compute the mean of those maxima per decade. This needs two rounds of `group_by()` and `summarise()`: first per election, then per decade. Hint: `decade = year %/% 10 * 10`, and drop the rows without a vote share before the first round.

```
largest_by_decade <- parl |>
  filter(!is.na(vote_share)) |>
  mutate(decade = year %/% 10 * 10) |>
  group_by(country_name, election_date, decade) |>
  summarise(max_vote = max(vote_share), .groups = "drop") |>
  group_by(decade) |>
  summarise(
    mean_max_vote = mean(max_vote),
    n_elections = n()
  )

largest_by_decade |>
  print(n = 13)
```

```
# A tibble: 13 x 3
  decade mean_max_vote n_elections
  <dbl>         <dbl>         <int>
1  1900           45.4             27
2  1910           40.0             39
3  1920           36.9             57
4  1930           39.4             48
5  1940           40.5             49
6  1950           40.7             66
7  1960           41.1             59
8  1970           38.6             73
9  1980           38.8             75
10 1990           34.5            104
11 2000           34.7             98
12 2010           32.6            110
13 2020           30.6             37
```

Yes, the largest party is getting smaller. From the 1940s to the 1960s it won about 40 percent of the vote on average, and since the 2010s it has won only a little over 30 percent. Party systems have fragmented, with more parties sharing the vote. Without the `filter(!is.na(vote_share))`, elections where every vote share is missing give `max()` nothing to work with, it returns `-Inf`, and the decade means turn into `-Inf` too.

6 Your pipeline: a data summary

! Important

This section is step 2 of the workflow, the data summary. A data summary answers a precise question with a table a reader can understand without your code. From now on, every analysis you write starts with one.

The question. In national parliament elections since 2000, did the average party of each family win a larger vote share in EU member states than outside?

1. Write **one pipeline**, starting from `parl_family`, that:

- keeps the elections from 2000 onwards
- joins the fixed `eu_members`
- adds `eu_member`, which is `TRUE` if the country was a member **on the day of the election**: `!is.na(accession_date) & accession_date <= as.Date(election_date)`
- drops the two values of `family_name` that are not families
- returns one row per family and membership status, with the mean vote share in a column `mean_vote`

Assign the result to `eu_families`, and run the pipeline after every new line, as in class.

```
eu_families <- parl_family |>
  filter(year >= 2000) |> # rows
  left_join(eu_members, by = join_by(country_name == country)) |> # add a table
  mutate(
    eu_member = !is.na(accession_date) &
      accession_date <= as.Date(election_date)
  ) |> # new column
  filter(!family_name %in% c("no family", "to be coded")) |> # real families
  group_by(family_name, eu_member) |> # groups
  summarise(mean_vote = mean(vote_share, na.rm = TRUE), .groups = "drop")

eu_families
```

```
# A tibble: 18 x 3
  family_name      eu_member mean_vote
  <chr>            <lgl>      <dbl>
1 Agrarian         FALSE      12.8
2 Agrarian         TRUE       11.3
3 Christian democracy FALSE      8.58
4 Christian democracy TRUE      13.0
5 Communist/Socialist FALSE      6.15
6 Communist/Socialist TRUE       6.43
7 Conservative     FALSE      17.4
8 Conservative     TRUE      13.1
9 Green/Ecologist  FALSE      6.65
10 Green/Ecologist  TRUE       4.97
11 Liberal          FALSE      9.52
12 Liberal          TRUE      10.0
13 Right-wing       FALSE      5.61
14 Right-wing       TRUE       7.75
15 Social democracy FALSE      16.7
16 Social democracy TRUE      16.3
17 Special issue    FALSE      3.70
18 Special issue    TRUE       2.32
```

2. Turn `eu_families` into a table for a reader: `pivot_wider()` so that each family is one row, with one column for members and one for non-members. Then pass it to `knitr::kable()`, with `digits = 1` and readable column names. The cell below already carries the label `tbl-eu-families`. Replace its placeholder `tbl-cap` with a caption a reader can understand without the text: what is in the table, for which elections, and in which unit.

```
eu_families |>
  pivot_wider(names_from = eu_member, values_from = mean_vote) |>
  select(family_name, `TRUE`, `FALSE`) |>
  knitr::kable(
    digits = 1,
    col.names = c("Party family", "EU member", "Not an EU member")
  )
```

Table 1: Mean vote share of a party, in percent, by party family and by whether the country was an EU member on election day. National parliament elections from 2000 to 2023 in the 37 ParlGov democracies. Rows without a party family are excluded. Data: ParlGov (Döring and Manow 2024) and the course’s EU membership file.

Party family	EU member	Not an EU member
Agrarian	11.3	12.8
Christian democracy	13.0	8.6
Communist/Socialist	6.4	6.1
Conservative	13.1	17.4
Green/Ecologist	5.0	6.7
Liberal	10.0	9.5
Right-wing	7.8	5.6
Social democracy	16.3	16.7
Special issue	2.3	3.7

3. Below the table, write three to four sentences that answer the question.

- Reference the table with `@tbl-eu-families`.
- Put at least one number into a sentence with inline code, so that it updates when the data does.
- Name one thing the mean hides, or one reason why a gap between members and non-members is not yet an effect of EU membership.

There is no single answer: it depends on the family, as Table 1 shows. Christian democratic parties won 13.0 percent on average inside the EU against 8.6 percent outside, and right-wing parties also did better inside. Conservative, Green and agrarian parties did better outside the EU, and social democrats did about equally well in both groups. The mean hides how many parties a family runs in a country: where a family is split into several small parties, its average party is small, and the majoritarian democracies outside the EU, such as Canada and Australia, have few and large parties. The gaps are therefore descriptive only, since members and non-members also differ in their electoral systems, their history and their region, so none of the gaps is yet an effect of EU membership.

Tip

Tidy for the computer, wide for the reader. Keep `eu_families` tidy, and pivot wider only in the cell that prints the table.

Reflection

Which step of this homework took you longest, and why? Is there a verb or a join you are still unsure about?

The reflection is personal and has no sample answer. A good one names a concrete step, such as the join that silently lost Slovakia, and says what made it hard.

Session info and execution time

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24 ucrt)
Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)
```

```
Matrix products: default
  LAPACK version 3.12.1
```

```
locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8
```

```
time zone: Europe/Berlin
tzcode source: internal
```

```
attached base packages:
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
[1] here_1.0.2      lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0
[5] dplyr_1.2.1     purrr_1.2.2     readr_2.2.0     tidyr_1.3.2
[9] tibble_3.3.1    ggplot2_4.0.3   tidyverse_2.0.0
```

```
loaded via a namespace (and not attached):
[1] utf8_1.2.6      generics_0.1.4   stringi_1.8.9    hms_1.1.4
[5] digest_0.6.39   magrittr_2.0.5   evaluate_1.0.5   grid_4.6.1
```

[9]	timechange_0.4.0	RColorBrewer_1.1-3	fastmap_1.2.0	cellranger_1.1.0
[13]	rprojroot_2.1.1	jsonlite_2.0.0	scales_1.4.0	cli_3.6.6
[17]	rlang_1.3.0	crayon_1.5.3	bit64_4.8.6	withr_3.0.3
[21]	yaml_2.3.12	otel_0.2.0	tools_4.6.1	parallel_4.6.1
[25]	tzdb_0.5.0	curl_8.0.0	vctrs_0.7.3	R6_2.6.1
[29]	lifecycle_1.0.5	bit_4.6.0	vroom_1.7.1	pkgconfig_2.0.3
[33]	pillar_1.11.1	gtable_0.3.6	glue_1.8.1	xfun_0.60
[37]	tidyselect_1.2.1	knitr_1.52	farver_2.1.2	htmltools_0.5.9
[41]	rmarkdown_2.32	compiler_4.6.1	S7_0.2.2	readxl_1.5.0

```

end_time <- Sys.time()
cat(
  "Total execution time:",
  round(difftime(end_time, start_time, units = "mins"), 1),
  "minutes\n"
)

```

Total execution time: 0 minutes